

DEV2 – JAVL – Laboratoire Java**TD 07 – package, static****Résumé**

Cette séance revient sur la notion de *packages* et l'utilisation du mot-clef `static`.
Nous vous recommandons de tester sur votre machine tous les codes présentés ici.

Table des matières

1	La notion de package	1
2	Le mot-clef <code>static</code>	3
3	Mise en pratique	4

1 La notion de package

Les packages permettent de regrouper, ranger, des classes liées sémantiquement entre elles et leur donnent un nom unique.

Le nom complet (*qualified name*) d'une classe est composé du nom du package suivi du (petit) nom de la classe, par exemple : `java.util.ArrayList` ou `java.lang.String`.

Ainsi il est possible d'avoir 2 classes portant le même nom, par exemple `App`, pourvu qu'elles ne se trouvent pas dans le même package.

Built-in packages

Java fournit une série de packages pré-définis :

- ▷ `java.lang` : ce paquet contient les classes de base du langage java comme la classe `String`, `System`, `Math` ou encore la classe `Exception`.
- ▷ `java.util` : contient des classes implémentant des structures de données telles que les listes `ArrayList`, les dictionnaires `Dictionary` ou encore des classes permettant de gérer les dates `Date` ou encore la classe `Scanner` que vous avez déjà rencontrée.
- ▷ `java.io` et `java.nio` : contiennent des classes permettant de gérer les entrées / sorties comme la lecture ou l'écriture de fichiers.

La documentation complète des différents packages de base se trouve ici : <https://docs.oracle.com/en/java/javase/21/docs/api/java.base/module-summary.html>

Utiliser une classe

Pour utiliser la classe `java.util.Scanner`, il est possible :

- ▷ d'utiliser son nom complet / qualifié :

```
java.util.Scanner keyboard = new java.util.Scanner(...);
```

- ▷ d'utiliser `import` et le nom court de la classe

```
import java.util.Scanner;  
// [cut]  
Scanner keyboard = new Scanner(...);
```

C'est la deuxième manière de faire qui sera privilégiée.

Remarquez que l'instruction `import xxx` est une simple facilité syntaxique, et rien de plus. Cela évite de devoir utiliser les noms qualifiés des classes à chaque utilisation. Cela rendrait en effet le code illisible.

Les *imports* se trouvent au début du fichier, juste après la directive indiquant le *package*.

User-defined packages

Les développeurs peuvent également définir leurs propres *packages*.

Pour préciser qu'une classe fait partie d'un package, on utilise le mot clé `package` comme *première ligne* de la classe.

```
package g12345.scrabble;  
  
public class App {  
    // insert code here  
}
```

Un nom de package s'écrit entièrement en minuscules et correspond généralement à un nom de domaine inversé (une adresse internet).

Pour nous ce devrait être `be.he2b.esi.g12345.<mon projet>...` mais l'on se contente de `g12345.scrabble`.

Les classes sont rangées dans des répertoires correspondants au nom de leur package.

Ainsi la classe `g12345.scrabble.Dictionary` se trouvera dans un répertoire `scrabble` qui lui même se trouvera dans un répertoire `g12345`. Vous pouvez le constater en allant *farfouiller* dans les fichiers de votre projet.

Exercice 1 Mise en pratique : un (vrai) dictionnaire

Téléchargez le fichier <http://www.3zsoftware.com/listes/ods6.zip> et extrayez-en le fichier `ods6.txt`, il s'agit du dictionnaire Officiel Du Scrabble 2012.

Placez ce dernier dans le répertoire `resources` (`src > main > resources`) de votre projet Scrabble.

Dans le *package* `g12345.scrabble`, créez une classe `Dictionary` et placez-y le code ci-dessous que vous lisez attentivement. Cela permet de charger en mémoire le contenu du fichier.

Ajoutez les *imports* nécessaires.

```

package g12345.scrabble;
[...]
public class Dictionary {

    private List<String> words;

    public Dictionary() {
        this.words = new ArrayList<>();
        String filename = "ods6.txt";
        System.out.println("loading dictionary from file: "+filename);
        InputStream is = getClass().getClassLoader()
            .getResourceAsStream(filename);
        Scanner scan = new Scanner(is);
        while(scan.hasNext()) {
            words.add(scan.next()); // ajoute chacun des mots du fichier dans la liste.
        }
        System.out.println("dictionary loaded");
    }
}

```

Ajoutez une méthode `boolean contains(String word)` qui vérifie si le mot passé en paramètre appartient au dictionnaire. Attention les mots du fichier sont en majuscule, assurez-vous que les mots en minuscule sont également acceptés. Pour cela trouvez la méthode de la classe `String` qui permet de mettre un mot en MAJUSCULE.

Testez votre dictionnaire dans une méthode `main` avec différents mots.

Nous utiliserons cette classe à la fin du TD pour compléter votre jeu de Scrabble.

2 Le mot-clef `static`

Le mot clef `static` permet d'associer un attribut ou une méthode à la classe directement et non plus à une *instance* de la classe. On peut voir cela comme des variables ou des méthodes *globales*.

Attributs

```

public class MathUtil {
    /**
     * Pi. For most calculations, NASA uses 15 digits, we use 16.
     */
    static double pi = 3.1415926535897932;
    /**
     * Euler's number
     */
    static double e = 2.718281828459045;
}

```

À partir d'une autre classe on y accède avec le nom de la classe suivi du nom de l'attribut.

```

public class Main {
    public static void main(String[] args) {
        System.out.println(MathUtil.pi);
        System.out.println(MathUtil.e);
    }
}

```

Vous remarquerez qu'il n'y a pas d'instanciation de la classe `MathUtil`. Pas besoin de créer un nouvel objet `MathUtil util = new MathUtil()` pour utiliser `pi` et `e`.

En mémoire, les valeurs de `pi` et de `e` ne seront stockées qu'une seule fois. Si elle n'était pas `static` elle serait stockée autant de fois que d'instances de cette classe auront été créées.

Méthodes

Il en va de même pour les méthodes.

Ajoutez la méthode suivante dans votre classe `MathUtil` et testez là dans votre `main`. Notez qu'à l'intérieur même de la classe `MathUtil`, on peut directement accéder à `pi` sans devoir y mettre le nom de la classe.

```
static double circleArea(double radius) {  
    return pi*radius*radius;  
}
```

Constantes

En Java une constante est une variable globale (une variable de classe) qui ne peut-être modifiée, pour s'en assurer on utilise le mot-clef `final`.

Par convention, le nom des constantes est toujours écrit entièrement en MAJUSCULES. Si le nom est composé de plusieurs mots on les sépare par des tiret-du-bas (*underscore*), par exemple, `SERVER_IP` ou `MAX_SIZE`.

Les variables `PI` et `E` sont clairement des constantes. On les déclarera donc plutôt comme-ci :

```
public static final double PI = 3.141592653589793;  
  
public static final double E = 2.718281828459045;
```

3 Mise en pratique

Exercice 2 MathUtil

Complétez le classe `MathUtil` en lui ajoutant les méthodes suivantes :

- ▷ `double circlePerimeter(double radius)` qui calcule le périmètre d'un cercle de rayon donné. Pour rappel le périmètre s'obtient par la formule $2\pi r$.
- ▷ `double ellipseArea(double a, double b)` qui calcule l'aire d'une ellipse de grand axe `a` et de petit axe `b`. Pour rappel cette aire s'obtient par la formule $ab\pi$.

Exercice 3 Scrabble on ne place pas n'importe quoi !

Améliorer votre jeu Scrabble comme suit :

- ▷ Créez une constante (`BOARD_SIZE`) pour la hauteur et la largeur de votre plateau de jeu de valeur 15. Utilisez cette constante partout dans le code où cela est pertinent.
- ▷ Modifier la variable `filename` de la classe `Dictionary` afin d'en faire une constante de classe.
- ▷ Utilisez la classe `Dictionary` ci-dessus afin de vérifier que les mots choisis par le joueur s'y trouvent.